

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1702

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _Vijiya Priya S.A (192524417) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

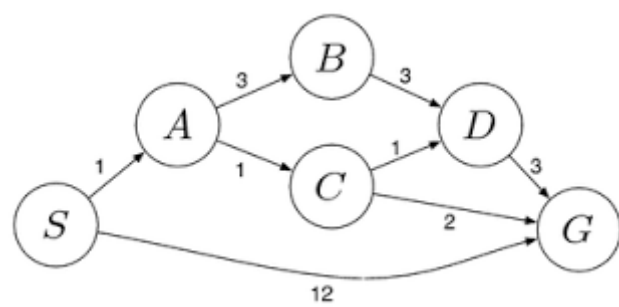
Signature of the student:

Vijiya Priya S.A
192524417

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.
The delivery network is represented as a weighted graph:



ITA1702

ARTIFICIAL INTELLIGENCE

CO1-ASSESSMENT TOOL 1

Annexure A: Analytical Problem Solving

Problem 1: Water Jug Problem

Problem formulation

Goal: obtain exactly **2 gallons in the 4-gallon jug**, using a 4-gallon jug and a 3-gallon jug with no markings.

- vi. **State:** an ordered pair (x, y) , where x is the water in the 4-gallon jug ($0 \leq x \leq 4$) and y is the water in the 3-gallon jug ($0 \leq y \leq 3$).
- vii. **Initial state:** $(0, 0)$.
- viii. **Goal state:** $(2, y)$ for any y ; that is, $x = 2$.
- ix. **Actions:** fill a jug from the pump, empty a jug on the ground, or pour from one jug into the other until the source is empty or the destination is full.

Production rules

No.	Condition	Action	Resulting state
1	$x < 4$	Fill the 4-gallon jug	$(4, y)$
2	$y < 3$	Fill the 3-gallon jug	$(x, 3)$
3	$x > 0$	Empty the 4-gallon jug on the ground	$(0, y)$

No.	Condition	Action	Resulting state
4	$y > 0$	Empty the 3-gallon jug on the ground	$(x, 0)$
5	$x + y \geq 4$ and $y > 0$	Pour from 3-gal into 4-gal until the 4-gal is full	$(4, y - (4 - x))$
6	$x + y \geq 3$ and $x > 0$	Pour from 4-gal into 3-gal until the 3-gal is full	$(x - (3 - y), 3)$
7	$x + y \leq 4$ and $y > 0$	Pour all of the 3-gal into the 4-gal	$(x + y, 0)$
8	$x + y \leq 3$ and $x > 0$	Pour all of the 4-gal into the 3-gal	$(0, x + y)$

Solution (found by breadth-first search over the state space)

Step	State (x, y)	Action	Rule
0	(0, 0)	Start: both jugs empty	-
1	(0, 3)	Fill the 3-gallon jug from the pump	2
2	(3, 0)	Pour the 3-gallon jug into the 4-gallon jug	7
3	(3, 3)	Fill the 3-gallon jug again	2
4	(4, 2)	Pour from the 3-gallon jug into the 4-gallon jug until it is full (1 gallon moves, 2 remain)	5
5	(0, 2)	Empty the 4-gallon jug on the ground	3
6	(2, 0)	Pour the 2 gallons from the 3-gallon jug into the 4-gallon jug	7

Result: the 4-gallon jug holds exactly **2 gallons** after 6 moves. An equally short alternative is: $(0,0) \rightarrow (4,0) \rightarrow (1,3) \rightarrow (1,0) \rightarrow (0,1) \rightarrow (4,1) \rightarrow (2,3)$. Both reach the goal in 6 steps, and BFS confirms that no shorter solution exists.

Problem 2: Mars Rover as an Intelligent Agent

The Mars Rover is an autonomous agent that moves across the Martian surface, collects soil and rock samples, analyses them with onboard instruments and transmits findings to Earth.

i. Percepts

- x. Camera images (terrain, rocks, obstacles, targets of scientific interest).
- xi. Terrain and navigation data: wheel odometry, gyroscope/IMU (tilt, slope), wheel slip, GPS-like localisation from visual landmarks.
- xii. Proximity and hazard sensors: obstacle detection, cliff or crater edges.
- xiii. Environmental sensors: temperature, wind, dust, radiation, sunlight level.
- xiv. Sample analysis readings: spectrometer, chemical and mineral composition, moisture or organic compound indicators.
- xv. Internal status: battery level, power from solar panels, motor and instrument health, memory and communication link status.
- xvi. Commands and schedules received from Earth mission control.

ii. Operating environment (characteristics)

Property	Value	Reason
Observability	Partially observable	Sensors see only nearby terrain; hidden rocks, soil strength and far areas are unknown.
Agents	Single agent (mostly)	One rover acts, though Earth control and orbiters cooperate with it.
Determinism	Stochastic	Wheel slip, dust, slopes and sensor noise make action outcomes uncertain.
Episodic vs sequential	Sequential	Present decisions (path, power use) affect future options.
Static vs dynamic	Dynamic	Dust storms, lighting and temperature change while the rover is deciding.

Property	Value	Reason
Discrete vs continuous	Continuous	Position, speed, time and sensor values are continuous.
Known vs unknown	Partly unknown	Laws of physics are known, but the local terrain is not.

iii. Actions

- xvii. Move: drive forward or backward, turn, climb, stop, and steer around obstacles.
- xviii. Manipulate: deploy the robotic arm, drill, scoop or scrape samples, place samples in instruments.
- xix. Sense and analyse: capture images, run the spectrometer, perform chemical tests.
- xx. Communicate: transmit data to Earth via orbiter or antenna, receive commands.
- xxi. Manage resources: recharge using solar panels, enter sleep or safe mode, heat components.

iv. Performance measure

- xxii. **Science value:** number and quality of samples analysed; discoveries of scientific significance; area explored.
- xxiii. **Safety:** rover survival, no collisions, no getting stuck or tipping over.
- xxiv. **Efficiency:** energy and time used per task; data successfully transmitted.
- xxv. **Reliability and mission duration:** system uptime, number of mission days completed, accuracy of analysis.

These may be combined as a weighted score, for example: $\text{Performance} = w_1 \cdot (\text{scientific value}) + w_2 \cdot (\text{distance/area covered}) - w_3 \cdot (\text{energy used}) - w_4 \cdot (\text{risk incurred})$.

v. Most suitable agent architecture

A hybrid, layered architecture combining model-based, goal-based and utility-based agents, with a learning component is most suitable:

- xxvi. **Reflex layer:** immediate responses such as stopping at a cliff edge, tilt limit exceeded or overheating (communication delay of 4 to 24 minutes to Earth makes remote control impossible for emergencies).
- xxvii. **Model-based layer:** the environment is partially observable, so the rover keeps a map, its position and a model of terrain and power state.
- xxviii. **Goal-based layer:** plans to reach a target rock, collect a sample and return to communication range.
- xxix. **Utility-based layer:** trades off science value against energy, time and risk when choosing between targets or routes.
- xxx. **Learning element:** improves terrain classification and navigation from experience.

A purely reflex agent could not plan or survive in unseen terrain, and a purely goal-based agent could not weigh risk against reward, so the hybrid agent is the best choice.

Problem 3: The 8-Queens Problem

Task: place 8 queens on an 8×8 chessboard so that no two queens attack each other, meaning no two share a row, a column or a diagonal. The configuration in the question is not a solution because the queens in the first and last columns lie on the same diagonal.

Problem formulation

Component	Definition
States	Incremental formulation: any arrangement of 0 to 8 queens on the board, with queens placed column by column. Choosing only non-attacking placements reduces the state space to 2,057 states.
Initial state	Empty board (no queens).
Actions	Add a queen to the leftmost empty column, in a square that is not attacked by any queen already placed.
Transition model	Returns the board with the new queen added.
Goal test	8 queens are on the board and none is attacked.

Component	Definition
Path cost	Not relevant; only the final configuration matters (every step costs 1).

Alternative (complete-state) formulation: begin with all 8 queens placed (one per column) and move a queen within its column to reduce conflicts. State = a vector $(r_1, \dots, r_8)$, where r_i is the row of the queen in column i . Counting states shows why the formulation matters: placing 8 queens on any squares gives $64 \times 63 \times \dots \times 57 \approx 1.8 \times 10^{14}$ sequences, while allowing only one queen per column gives $8^8 \approx 16.7$ million states, and requiring a permutation (one queen per row and per column) gives $8! = 40,320$ states.

Exploring the search space with different strategies

Strategy	How it works on 8-Queens	Evaluation
Breadth-first search	Expands all placements of k queens before $k+1$ queens.	Complete, but memory grows exponentially; impractical.
Depth-first search / backtracking	Places a queen column by column, and when no safe row exists, backtracks to the previous column.	Memory-efficient; finds a solution quickly because depth is only 8. Standard method for this problem.
Constraint checking (forward checking)	Removes attacked squares from later columns immediately, and backtracks when a column has no safe squares.	Prunes the search tree strongly; far fewer nodes than plain DFS.
Hill climbing (complete-state)	Moves one queen within its column to the position that most reduces the number of attacking pairs (heuristic h).	Fast, but gets stuck at local minima (about 86% of random starts fail); random restarts fix this.
Min-conflicts / simulated annealing	Picks a conflicted queen and moves it to the row with fewest conflicts (with occasional random moves in annealing).	Solves even very large N-Queens instances quickly.

Example solution (row number of the queen in columns 1 to 8): **(1, 5, 8, 6, 3, 7, 2, 4)**. No two queens share a row, and no two lie on the same diagonal, so it is a valid solution. In total, the 8-Queens problem has **92 solutions** (12 distinct ones when rotations and reflections are treated as identical).

Conclusion: the incremental formulation with backtracking (with constraint checking) is the most efficient systematic approach, while min-conflicts is the best local search approach.

Problem 4: OLA Cab Booking Using a Problem-Solving Agent

Type of agent

The booking task is best handled by a **goal-based problem-solving agent with a utility-based component**:

- xxxi. **Goal-based problem-solving agent:** the goal is clear (reach the destination from the source). The agent formulates the goal, formulates the problem (states, actions, costs), searches for a solution and executes it (assign cab, drive route).
- xxxii. **Utility-based element:** the customer can choose among mini, micro, sedan, shared, prime and so on, so there are several ways to reach the same goal. The agent ranks the options with a utility function based on fare, waiting time, comfort and travel time, and recommends or selects the best match for the customer's preference.

Problem formulation

Component	Definition
Initial state	Customer at pickup location, no cab booked.
Goal state	Customer delivered at destination.
Actions	Enter source and destination, select cab type, confirm booking, assign driver, start trip, follow route, end trip, pay.
Path cost	Fare + waiting time + travel time (weighted), possibly with a penalty for lower comfort.

Component	Definition
Percepts	Locations, available cabs and drivers, cab types and fares, traffic, customer preference.

Pseudocode

```

function OLA_BOOKING_AGENT(percept) returns action
    persistent: state, plan (a sequence of actions), preference

    state <- UPDATE_STATE(state, percept)    // location, traffic, cabs
    if plan is empty then
        goal <- FORMULATE_GOAL(state)        // reach destination D from source S
        problem <- FORMULATE_PROBLEM(state, goal)
        plan <- SEARCH(problem)              // see below
        if plan = failure then return "No cab available"
    action <- FIRST(plan); plan <- REST(plan)
    return action

function SEARCH(problem)
    options <- GET_AVAILABLE_CABS(source, cab_types) // mini, micro, sedan, shared, prime
    best <- null; best_utility <- -infinity
    for each cab in options do
        fare <- ESTIMATE_FARE(cab, source, destination)
        wait <- ESTIMATE_PICKUP_TIME(cab)
        time <- ESTIMATE_TRAVEL_TIME(cab, route)
        U <- w1*Comfort(cab) - w2*fare - w3*wait - w4*time
        if cab matches customer preference then U <- U + bonus
        if U > best_utility then best <- cab; best_utility <- U
    if best = null then return failure
    return [SELECT(best), CONFIRM_BOOKING, ASSIGN_DRIVER, TRACK_PICKUP,
           START_TRIP, FOLLOW_ROUTE(S, D), END_TRIP, PAY]

// Main loop
while customer not at destination do
    percept <- GET_PERCEPT()
    action <- OLA_BOOKING_AGENT(percept)

```

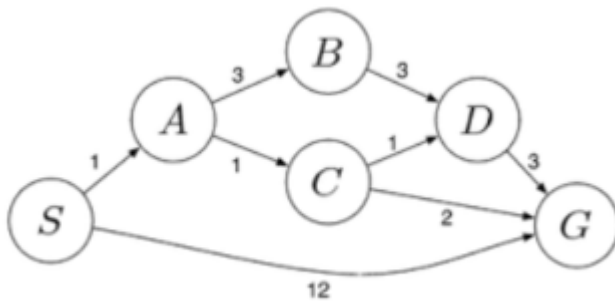
```
EXECUTE(action)
```

```
if cab cancelled or traffic blocks route then plan <- empty // replan
```

If the customer picks a cab type explicitly, the utility step is skipped and the agent searches only within the chosen category. If the chosen cab is unavailable, the agent re-runs the search and offers the next-best option.

Problem 5: Least-Cost Delivery Route Using Uniform Cost Search

The delivery network (directed weighted graph) is given below:



From	To	Cost
S	A	1
S	G	12
A	B	3
A	C	1
B	D	3
C	D	1
C	G	2
D	G	3

UCS algorithm

UCS keeps a priority queue (frontier) ordered by the cumulative path cost $g(n)$ from S. It repeatedly removes the cheapest node, stops when that node is the goal, and otherwise expands it by adding its neighbours with updated costs. If a cheaper path to a node already in the frontier is found, the cost is updated.

Step-by-step trace

Step	Node expanded (g)	Nodes added or updated	Frontier after step (node: cost)
1	S (0)	A = 1, G = 12	A:1, G:12
2	A (1)	B = 1+3 = 4, C = 1+1 = 2	C:2, B:4, G:12
3	C (2)	D = 2+1 = 3, G = 2+2 = 4 (improves 12)	D:3, B:4, G:4
4	D (3)	G via D = 3+3 = 6 (not better than 4, ignored)	B:4, G:4
5	B (4)	D via B = 4+3 = 7 (not better than 3, ignored)	G:4
6	G (4)	Goal test passed. Search stops.	empty

Order of expansion: **S** → **A** → **C** → **D** → **B** → **G**. (B and G tie at cost 4; expanding either first does not change the result.)

Result

Item	Answer
Least-cost path	S → A → C → G
Total cost	4 (1 + 1 + 2)

Item	Answer
Other paths for comparison	$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 6$; $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 10$; $S \rightarrow G = 12$

The direct route $S \rightarrow G$ looks shortest in number of edges but has the highest cost (12). UCS correctly ignores edge count and chooses by cost, which guarantees the optimal route. UCS is complete and optimal when all edge costs are positive.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem